

Complete x86-64 Assembly Manual (Intel Syntax)

NECKER

December 3, 2025

Indice

1	Registers in x86-64	2
1.1	General-Purpose Registers (GPR)	2
1.2	Schematic Example of Register Overwriting	3
1.2.1	Execution Result (Visual Output)	3
1.3	Implicit Zero-Extension Rule	4
1.4	Special Registers	4
2	Memory Measurement Units	4
3	Fundamental Instructions	4
3.1	Data Movement and Stack	4
3.2	Address Arithmetic (LEA)	5
3.3	Basic Arithmetic	5
3.4	Multiplication and Division	5
3.5	Logical and Bitwise Operations	5
3.6	Flow Control	6
3.7	Shift and Rotation Instructions (Bitwise)	7
4	Accessing Registers via Addresses	8
5	Syscalls	9
5.1	Extra Useful Information	12
5.2	Definition of Opcode	12
5.3	Pitfall 1: Variable Instruction Length	13
5.4	Pitfall 2: Correct Addressing and Sizing	13
6	Instruction Sizes	13

1 Registers in x86-64

Registers are high-speed storage units located inside the CPU. In x86-64, there are several types: General-Purpose Registers (GPR), segment registers, control registers, and special registers. Here we focus on the most important ones for basic Assembly programming.

1.1 General-Purpose Registers (GPR)

There are 16 general-purpose 64-bit registers. Each has sub-registers to access smaller portions of the bits. Writing to a sub-register can affect the entire register, according to specific rules.

64-bit Register	32-bit	16-bit	Low 8-bit	High 8-bit	Typical Description
RAX	EAX	AX	AL	AH	Accumulator: Used for arithmetic and function return values.
RBX	EBX	BX	BL	BH	Base: For indices and general pointers.
RCX	ECX	CX	CL	CH	Counter: Used in loops and bit shifts.
RDX	EDX	DX	DL	DH	Data: Extension for multiplication/division operations.
RSI	ESI	SI	SIL	-	Source Index: For strings and arrays (source).
RDI	EDI	DI	DIL	-	Destination Index: For strings and arrays (destination).
RBP	EBP	BP	BPL	-	Base Pointer: For stack frames inside functions.
RSP	ESP	SP	SPL	-	Stack Pointer: Points to the top of the stack.
R8	R8D	R8W	R8B	-	Extended general-purpose register (from R8 to R15).
R9	R9D	R9W	R9B	-	Extended general-purpose register.
R10	R10D	R10W	R10B	-	Extended general-purpose register.
R11	R11D	R11W	R11B	-	Extended general-purpose register.
R12	R12D	R12W	R12B	-	Extended general-purpose register.
R13	R13D	R13W	R13B	-	Extended general-purpose register.
R14	R14D	R14W	R14B	-	Extended general-purpose register.
R15	R15D	R15W	R15B	-	Extended general-purpose register.
Note: Writing to a large register (RDI) always overwrites its smaller portions (EDI, DIL).					

Tabella 1: Complete hierarchy of General-Purpose Registers (GPR) in x86-64.

Notes on sub-registers:

- Registers from RAX to RDX have high 8-bit sub-registers (AH, BH, etc.), but those from RSI onwards do not.
- Extended registers (R8-R15) are only available in 64-bit mode.

- Registers and sub-registers reference the same physical memory cell; therefore, writing to a 64-bit register overwrites all its sub-registers, and vice versa, writing to a sub-register modifies the upper register up to the maximum bit overwritten.

1.2 Schematic Example of Register Overwriting

Registers and sub-registers share the same physical memory area within the CPU. Writing to a 64-bit register (such as **RAX**) completely overwrites all associated sub-registers. Conversely, writing to a sub-register (such as **AL** or **EAX**) modifies only that specific portion of bits, leaving the others intact (with the exception of the implicit zero-extension rule for 32-bit writes, which zeroes out bits 32-63).

To illustrate this visually, we will use a Python simulation that emulates the behavior of the **RAX** register. The code can be executed in a Python environment to verify the results. Below are the simulator code and the generated output.

1.2.1 Execution Result (Visual Output)

Below is the output generated by running the script above. It shows how each operation modifies the specific bits of the register.

```
Initial state:
RAX: 0x0000000000000000
EAX: 0x00000000
AX: 0x0000
AH: 0x00
AL: 0x00

After mov rax, 0x1122334455667788:
RAX: 0x1122334455667788
EAX: 0x55667788
AX: 0x7788
AH: 0x77
AL: 0x88

After mov al, 0x99:
RAX: 0x1122334455667799
EAX: 0x55667799
AX: 0x7799
AH: 0x77
AL: 0x99

After mov eax, 0xAABBCCDD:
RAX: 0x00000000AABBCCDD
EAX: 0xAABBCCDD
AX: 0xCCDD
AH: 0xCC
AL: 0xDD

After mov ah, 0xEE:
RAX: 0x00000000AABBEEDD
EAX: 0xAABBEEDD
AX: 0xEEDD
AH: 0xEE
AL: 0xDD

After mov ax, 0xFF00:
RAX: 0x00000000AABBFF00
EAX: 0xAABBFF00
AX: 0xFF00
AH: 0xFF
```

AL: 0x00

1.3 Implicit Zero-Extension Rule

When writing to a sub-register, the behavior regarding the upper bits varies:

- **32-bit Write** (e.g., `EAX: mov EAX, EBX`): Automatically zeroes out bits 32-63 of the 64-bit register.
- **16-bit or 8-bit Write** (e.g., `AX, AL`): Does not modify the bits above the 16/8 thresholds.

This rule is crucial for optimizations, such as calculating moduli without arithmetic.

1.4 Special Registers

1. **RIP (Instruction Pointer)**: Points to the current instruction. It is modified by jumps and calls.
2. **RFLAGS (Flags Register)**: Contains flags like ZF (Zero Flag), SF (Sign Flag), CF (Carry Flag), OF (Overflow Flag). Used to signal specific behaviors such as a result zeroing out after an addition, etc.
3. **Segment Registers**: CS, DS, ES, FS, GS, SS. In standard flat mode, they are not necessary for memory access.

2 Memory Measurement Units

In Assembly, data sizes are defined as follows:

Term	Size in Bytes	Size in Bits
Byte	1	8
Word	2	16
Double Word (Dword)	4	32
Quad Word (Qword)	8	64
Double Quad Word (DQword)	16	128

Tabella 2: Memory measurement units in x86-64.

3 Fundamental Instructions

Instructions are grouped by category. The syntax is `INSTRUCTION Destination, Source`, where Destination is often a register or memory location, and Source is a register, memory location, or immediate value.

3.1 Data Movement and Stack

These instructions manage data transfer and the stack (which grows downwards).

Instruction	Syntax	Explanation
MOV	MOV Dest, Source	Moves value from Source (register, memory, immediate) to Dest. Does not alter flags. E.g.: <code>MOV RAX, 42</code> .
PUSH	PUSH Source	Decrements RSP by 8, saves Source onto the stack. Used to preserve registers.
POP	POP Dest	Copies from the top of the stack to Dest, increments RSP by 8.

3.2 Address Arithmetic (LEA)

The LEA instruction calculates an effective address, but **does not access memory**. It is frequently used to perform fast arithmetic via the addressing syntax.

Instruction	Syntax	Explanation
LEA	LEA Dest, [Base + Index*Scale + Offset]	Computes the address given by the expression without reading its content. Used for arithmetic operations (multiplication by 2/4/8, additions), or to construct addresses. Does not modify flags.

3.3 Basic Arithmetic

These modify flags (CF, OF, SF, ZF, etc.).

Instruction	Syntax	Explanation
ADD	ADD Dest, Source	Dest = Dest + Source. Sets flags for overflow/carry.
SUB	SUB Dest, Source	Dest = Dest - Source. Sets flags.
INC	INC Dest	Dest = Dest + 1. Does not alter CF.
DEC	DEC Dest	Dest = Dest - 1. Does not alter CF.

3.4 Multiplication and Division

For signed numbers, use IMUL/IDIV.

Instruction	Syntax	Explanation
IMUL	IMUL Dest, Source1, Imm	Dest = Source1 * Imm (3 operands).
IMUL	IMUL Dest, Source	Dest = Dest * Source (2 operands).
IDIV	IDIV Divisor	(RAX) / Divisor. Quotient in RAX, remainder in RDX.

3.5 Logical and Bitwise Operations

These modify individual bits.

Instruction	Syntax	Explanation
AND	AND Dest, Source	Dest = Dest bitwise AND Source. Sets ZF if zero.
OR	OR Dest, Source	Dest = Dest bitwise OR Source.
XOR	XOR Dest, Source	Dest = Dest bitwise XOR Source. Commonly used to clear a register (XOR RAX, RAX → RAX=0).

Tabella 8: Truth Tables for Fundamental Logical Operations

A	B	A AND B	A OR B	A XOR B
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

3.6 Flow Control

These handle jumps and functions.

Instruction	Syntax	Explanation
CMP	CMP register1 register2	Sets the Zero Flag to 1 or 0 to tell conditional jumps whether to jump or not.
Unconditional Jump		
JMP	JMP Label / offset	Unconditional jump. Execution continues from the new address. Can be Absolute (direct address) or Relative (offset relative to RIP).
Conditional Jump (Flag Based)		
JE / JZ	JE Label	Jumps if Equal / Zero Flag is set (ZF=1).
JNE / JNZ	JNE Label	Jumps if Not Equal / Zero Flag is cleared (ZF=0).
JG / JNLE	JG Label	Jumps if Greater (ZF=0 and SF=0F). For signed numbers.
JA	JA Label	Jumps if Above (CF=0 and ZF=0). For unsigned numbers.
JL / JNGE	JL Label	Jumps if Less (SF $\neq$ 0F). For signed numbers.
JB	JB Label	Jumps if Below (CF=1). For unsigned numbers.
Function Calls and Return		
CALL	CALL Label	Calls a function. Saves the return address (current RIP) onto the Stack (PUSH RIP), then performs a JMP to Label.
RET	RET	Returns from a function. Pops the return address from the Stack (POP RIP) and jumps to that address.
Interfacing with the Operating System		
SYSCALL	SYSCALL	System Call . Halts execution and passes control to the kernel. The requested service is specified in RAX.

1. **Unconditional Jump (JMP):** Used to create an infinite loop.

```
.loop_start:
; Loop code...
jmp .loop_start    ; Jumps to label .loop_start
```

2. **Conditional Jump (JE) after Comparison:** Performs branching (IF/ELSE).

```
mov rax, 5
cmp rax, rdi      ; Compares RAX with RDI. Sets flags.
je .RDI_Equal_5   ; Jumps here if ZF=1 (if RAX == RDI)

; Code for condition RDI != 5
jmp .end_check

.RDI_Equal_5:
; Code for condition RDI == 5

.end_check:
; Continues execution
```

3. **Function Call and Return:** Establishes the execution flow for the subroutine.

```
; ... Main code ...
mov rdi, 10      ; Argument 1 (RDI)
mov rsi, 20      ; Argument 2 (RSI)
call sum_function
; RAX now contains the sum (30)
; ... Continues ...

sum_function:
; RDI = 10, RSI = 20
mov rax, rdi     ; RAX = 10
add rax, rsi     ; RAX = 10 + 20 = 30
ret             ; Returns to the point right after CALL
```

4. **System Call (SYSCALL):** Executes kernel operations (e.g., exiting the program).

```
; Exit program (Syscall n. 60 in Linux x86-64)
mov rax, 60      ; Syscall number (sys_exit)
mov rdi, 0       ; Exit code (0 = success)
syscall         ; Executes Kernel call
```

3.7 Shift and Rotation Instructions (Bitwise)

These instructions shift the bits of a register. They are used for fast division/multiplication by powers of two and for bit isolation.

Instruction	Syntax	Explanation
SHL	SHL Dest, Count	Logical Shift Left (Multiplication by 2^N). Shifts N bits to the left; inserts 0 on the right.
SHR	SHR Dest, Count	Logical Shift Right (Unsigned Division by 2^N). Shifts N bits to the right; inserts 0 on the left.

Byte Isolation Strategy (in a 64-bit register)

To isolate a byte B_n in a 64-bit register (RAX, RDI, etc.), a combination of SHL and SHR is used. The strategy is to first bring byte B_n to bit 63, and then shift it back to bit 0 to clear the upper bits.

Tabella 11: Byte Positions in a 64-bit Register (QWORD)

Byte	Name	Bit Position	Shift for B_0	Value
B_0	AL/R8B	7 – 0	0	Least Significant (Low Byte)
B_1	AH/R9B	15 – 8	8	
B_2	R10B	23 – 16	16	
B_3	R11B	31 – 24	24	
B_4	R12B	39 – 32	32	
B_5	R13B	47 – 40	40	
B_6	R14B	55 – 48	48	
B_7	R15B	63 – 56	56	Most Significant (High Byte)

Isolation Example (To isolate the 5th byte, B_4):

1. `mov rax, rdi` (Copies the value)
2. `shl rax, 24` (Shifts B_4 to bit 55)
3. `shr rax, 56` (Shifts B_4 to bit 0 and clears the rest)

The original value of B_4 is now the sole content inside AL (and RAX).

4 Accessing Registers via Addresses

Normally, CPU registers are manipulated directly through their symbolic names (**RAX**, **RDI**, etc.). However, registers can also be mapped to addresses within the memory space (Memory-Mapped Registers).

Instructions for Indirect Access

Memory access instructions are utilized to read or write the value of a "mapped register":

Instruction	Syntax	Explanation
MOV	MOV Dest, [Reg_Addr]	Reading: Moves the 64-bit value (QWORD) stored at the memory address representing the register into a destination register (Dest).
MOV	MOV [Reg_Addr], Source	Writing: Moves the value from the source register (Source) to the memory address representing the register.
ADD	ADD [Reg_Addr], Imm	Modification: Directly increments the value of the memory-mapped register with an immediate value (Imm).

Hypothetical Example

Assuming that the register **RDI** is mapped to address 0x405000:

- `mov rax, [0x405000]` → Reads the "simulated" value of **RDI** into **RAX**.
- `mov [0x405000], rbx` → Writes the value of **RBX** into the mapped address, modifying the "simulated" **RDI**.

5 Syscalls

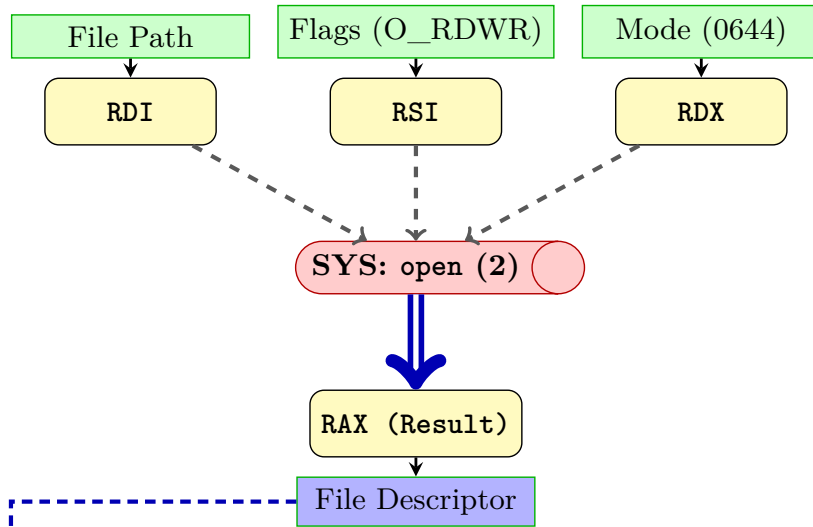
To work with syscalls in a Linux environment on x86-64 architecture, it is essential to know the mechanism for calling the kernel. Syscalls are invoked by placing the system call number in `rax`, the parameters in specific registers (`rdi`, `rsi`, `rdx`, etc.), and executing the `syscall` instruction. Upon return, `rax` contains the result (e.g., the return value or a negative error code if it failed).

Below is a summary table for common syscalls related to file management (open, read, write). The table lists the syscall number, primary registers, and typical values to set for basic scenarios (e.g., reading a text file).

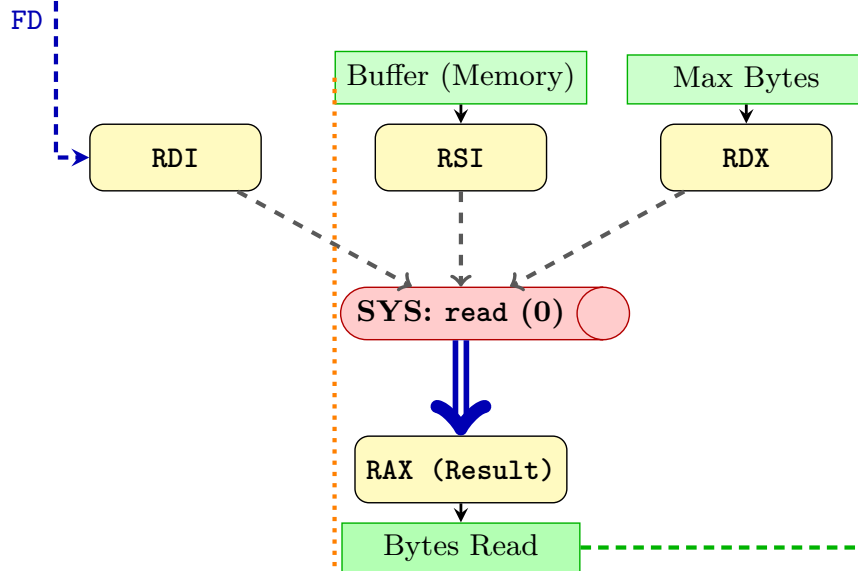
Syscall	Number (RAX)	Parameters (Registers)	Return Value (RAX)
open	2	• <code>RDI</code>: Address (pathname) of the null-terminated string of the file path. • <code>RSI</code>: Access flags (e.g., 0 for <code>O_RDONLY</code>, 1 for <code>O_WRONLY</code>). • <code>RDx</code>: Permissions (mode) for creation (used only with <code>O_CREAT</code>).	Success: Positive File Descriptor (FD). Error: Negative value (opposite of the error code).
read	0	• <code>RDI</code>: File Descriptor (FD) to read from. • <code>RSI</code>: Address of the buffer where read data should be saved. • <code>RDx</code>: Maximum number of bytes to read (count).	Success: Number of bytes read. EOF: 0. Error: Negative value.
write	1	• <code>RDI</code>: File Descriptor (FD) to write to (e.g., 1 for stdout). • <code>RSI</code>: Address of the buffer containing data to write. • <code>RDx</code>: Number of bytes to write (count).	Success: Number of bytes written. Error: Negative value.
sys_chmod	90	• <code>RDI</code>: Address (pathname) of the null-terminated string of the file path. • <code>RSI</code>: Mode, specified in octal (e.g., 0777 where the leading 0 indicates octal base, modifying permissions to r,w,x for everyone).	Success: 0

Syscall	Number (RAX)	Parameters (Registers)	Return Value (RAX)
			Error: Negative value (opposite of the error code).

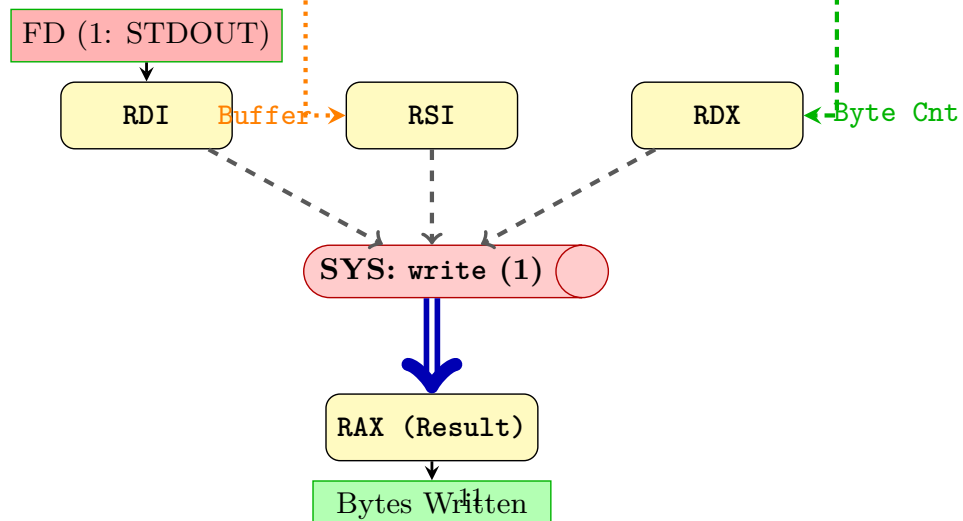
1. open()



2. read()



3. write()



5.1 Extra Useful Information

Here are general notes and best practices for using these syscalls in low-level assembly or shellcode:

- **File descriptors (fd):** These are integers representing open files. Standard values:
 - 0: stdin (standard input, e.g., keyboard).
 - 1: stdout (standard output, e.g., console output).
 - 2: stderr (standard errors).
 - $\text{fd} \geq 3$: Assigned dynamically by open (e.g., for user files like /flag).

To print, use `fd=1` in write to send output directly without an extra buffer.

- **Flags for open:** Define the opening mode. Common examples:
 - `O_RDONLY` (0): Read-only, ideal for files like a flag.
 - `O_WRONLY` (1): Write-only.
 - `O_RDWR` (2): Read and write.

Set with `xor rsi, rsi` to achieve 0 (null-free).

- **Error handling:** If `rax < 0` after a syscall, an error occurred (e.g., `-ENOENT` for file not found). In basic shellcodes, ignore it if you assume success, but in robust code check with `test rax, rax; js error`.
- **Buffer and stack:** For read/write, allocate space on the stack with `sub rsp, N` (e.g., 0x100 bytes). Use `mov rsi, rsp` for the pointer. Avoid overflow: `rdx` in read/write must match the allocated size.
- **Null-free and position-independent:** Under constraints like no 0x00 bytes, construct values using `xor/inc/shl` instead of direct `movs`. For strings, use little-endian format (e.g., inverted hex for dwords).

Opcodes, Instruction Encoding, and Common Pitfalls

5.2 Definition of Opcode

An **Opcode** (Operation Code) is the first field of any instruction in machine code and uniquely identifies the operation that the CPU must execute (e.g., `MOV`, `ADD`, `JMP`).

- An Opcode is usually a 1 or 2-byte hexadecimal code (e.g., `B8` for `MOV RAX, Imm64`).
- It is what the assembler translates from text (`mov`) into the binary code that the CPU understands.

Opcode Mapping for Fundamental Instructions

This table displays the Opcodes (hexadecimal codes) for the most frequently used types of instructions, particularly those relating to data movement and flow control.

Instruction	Syntax (Example)	Initial Opcode (H)	Typical Length (Bytes)
Data Transfer (MOV)			
MOV Reg → Reg	<code>mov rax, rbx</code>	48 89	3
MOV Imm → Reg 64-bit	<code>mov rax, 0x...</code>	48 B8	10
MOV Mem → Reg	<code>mov rax, [rdi]</code>	48 8B 07	3
MOV Mem Stack → Reg	<code>mov rdi, [rsp]</code>	48 8B 3C 24	4
Arithmetic and Logic			
ADD Reg → Reg	<code>add rax, rbx</code>	48 01	3
SUB Mem → Reg	<code>sub [rsp], rdi</code>	48 29	4
Flow Control			
JMP Relative 32-bit	<code>jmp Label</code>	E9	5 (1 Opcode + 4 Offset)
JMP Absolute (via Reg)	<code>jmp rax</code>	FF E0	2

Instruction	Syntax (Example)	Initial Opcode (H)	Typical Length (Bytes)
CALL Relative 32-bit	<code>call Label</code>	E8	5 (1 Opcode + 4 Offset)
Jcc (Conditional)	<code>je Label</code>	74	2 (1 Opcode + 1 Offset 8-bit)
Other Instructions			
NOP (No Operation)	<code>nop</code>	90	1

Note on Encoding (Prefixes 48): The prefix **48** (called **REX.W**) appears in many 64-bit instructions. This prefix informs the CPU that the operation must be executed on a 64-bit register (Quad Word) instead of the default 32-bit size, thereby modifying the base Opcode.

therefore, if we use 32-bit registers, the opcode will change, e.g., `mov eax, edx` without the 48 prefix.

5.3 Pitfall 1: Variable Instruction Length

In the x86-64 architecture, instructions do not have a fixed length. The length varies according to the Opcode, the number of operands, and the size of the operands (8, 16, 32, or 64 bits).

- **Example:** NOP occupies 1 byte (90). A relative JMP (32-bit jump) occupies 5 bytes (E9 + 4 offset bytes).
- **How to Avoid Errors:** In relative jumps (JMP, CALL), it is crucial to know the exact length of the instruction to correctly compute the destination offset. It is always preferable to use labels (`.label`) and leave the offset calculation to the assembler.

5.4 Pitfall 2: Correct Addressing and Sizing

The CPU needs to know exactly how many bytes to read or write from memory.

- **The POP/PUSH Pitfall:** The PUSH and POP instructions always modify the Stack Pointer register (RSP). To read or modify the value at the top of the stack **without** moving RSP, you must access memory directly:

```
mov rax, qword ptr [rsp]  (Read the content without changing RSP)
```

- **The QWORD PTR Pitfall:** If the destination is a register (e.g., `mov rax, [addr]`), the data size is implicit (64-bit, QWORD). If the destination is a memory address or the size is ambiguous, specifying the size with directives like QWORD PTR is mandatory:

```
sub qword ptr [rsp], rdi  (Operation on 8 bytes of memory)
```

6 Instruction Sizes

Clarification on the Length of MOV (RAX vs EAX)

The difference in the instruction length of MOV between 64-bit registers (RAX) and 32-bit registers (EAX) is entirely due to the size of the **Immediate Data** (the constant value being shifted) that the processor must load:

1. **mov rax, imm64 (10 bytes):** To load a complete 64-bit integer (up to $2^{64} - 1$) into RAX, the processor requires:
 - 1 Byte: REX prefix (required to enable the 64-bit operation).
 - 1 Byte: Opcode (0xB8).
 - 8 Bytes: The 64-bit **Immediate Data** to be loaded.
2. **mov eax, imm32 (5 bytes):** To load a 32-bit integer (up to $2^{32} - 1$) into EAX, the processor does not use the REX prefix for the immediate data but instead uses the native 32-bit encoding scheme:
 - 1 Byte: Opcode (0xB8).
 - 4 Bytes: The 32-bit **Immediate Data**.

Since shellcode is limited in length, programmers always avoid the expensive 10-byte instruction, preferring the 5-byte instruction if the value to be loaded fits within 32 bits, or constructing the value in RAX through shorter operations (such as `xor` and `add/push`).

Tabella 15: Byte Length of Common Instructions in Shellcode (x64)

Example Instruction	Typical Length	Details and Components
I. Data and Stack Manipulation		
<code>push reg</code> (e.g., <code>push rsp</code>)	1 Byte	Shortest instruction. Encodes the operation in a single byte (Opcode + implicit register since each register has its own Opcode for push).
<code>pop reg</code> (e.g., <code>pop rdi</code>)	1 Byte	Highly byte-efficient for recovering addresses. (Same as push)
<code>push imm32</code> (e.g., <code>push 0x...</code>)	5 Bytes	0x68 (Opcode) + 4 Bytes of immediate data.
<code>mov reg, reg</code> (e.g., <code>mov rdi, rax</code>)	3 Bytes	Requires a REX prefix (1B) to operate at 64-bit + Opcode/ModR/M (2B).
<code>inc reg</code> (e.g., <code>inc rax</code>)	3 Bytes	Requires a REX prefix (1B). Often replaced by <code>add reg, 1</code> in older encodings.
<code>nop</code>	1 Byte	No Operation (0x90). Used for padding.
II. Immediate Data and Jump		
<code>mov rax, imm64</code>	10 Bytes	Opcode 0xB8 + REX prefix (1B) + 8 Bytes of immediate data. (See clarification below).
<code>mov eax, imm32</code>	5 Bytes	Opcode 0xB8 + 4 Bytes of immediate data. Efficient if the value fits into 32 bits.
<code>xor reg, reg</code> (e.g., <code>xor rax, rax</code>)	3 Bytes	Extensively used to clear registers (RAX = 0) efficiently in terms of bytes.
III. Flow Control		
<code>syscall</code>	2 Bytes	Fixed instruction (0F 05) to execute kernel calls.
<code>jmp rel8</code>	2 Bytes	Unconditional relative jump with an 8-bit offset (if the jump is within 127 bytes, 1 offset byte = 8 bits = 127 choices, jumping by bytes). More compact than <code>jmp rel32</code> .
<code>jmp rel32</code>	5 Bytes	Unconditional relative jump with a 32-bit offset (same logic but with 4 offset bytes). More compact than <code>jmp rel64</code> .
<code>jmp rel64</code>	6+ Bytes	Unconditional relative jump with a 64-bit offset (depends on distance).
<code>jne/je rel8</code>	2 Bytes	Conditional jump with an 8-bit offset (e.g., after <code>cmp</code> or <code>test</code>).